Secure AI Systems: Red and Blue Teaming an MNIST Classifier

Team: *Shradha Sanjaykant Suryawanshi(CS25MTECH12018)*, *Suresh Kumar Baddipud (CS25M26P140002)*

Repository (CNN, attacks, adversarial/poisoned dataset generation, defenses): *https://github.com/<user>/<repo>*

Repository path	Contents
src/	CNN, training, metrics, integrity-checked model save/load, results summary
attacks/	Attack 1: adversarial examples (FGSM, PGD). Attack 2: backdoor data poisoning
defenses/	Adversarial training, feature squeezing, activation clustering
sast/	SAST script and a control sample of insecure patterns
results/	All metrics (JSON), summary.md, figures, SAST reports

1. System overview

The application trains a convolutional neural network (CNN) to classify 28×28 grayscale handwritten digits (0–9) from MNIST (60,000 training images, 10,000 test images).

Item	Value
Framework	PyTorch 2.14.1 (CPU build), torchvision 0.29.1
Attack library	IBM Adversarial Robustness Toolbox (ART) 1.20.1
Architecture	Conv(1→32, 3×3) → ReLU → Conv(32→64, 3×3) → ReLU → MaxPool 2×2 → Dropout 0.25 → FC(9216→128) → ReLU → Dropout 0.5 → FC(128→10)
Input handling	Pixels scaled to [0, 1]; normalisation (mean 0.1307, std 0.3081) happens inside the model, so attacks and defenses work in plain pixel space
Training	Adam, learning rate 0.001, batch size 128, 5 epochs, cross-entropy loss, seed 42
Hardware	Intel Xeon E5-1650 v4 (6 cores / 12 threads), no GPU

Data flow and trust boundaries

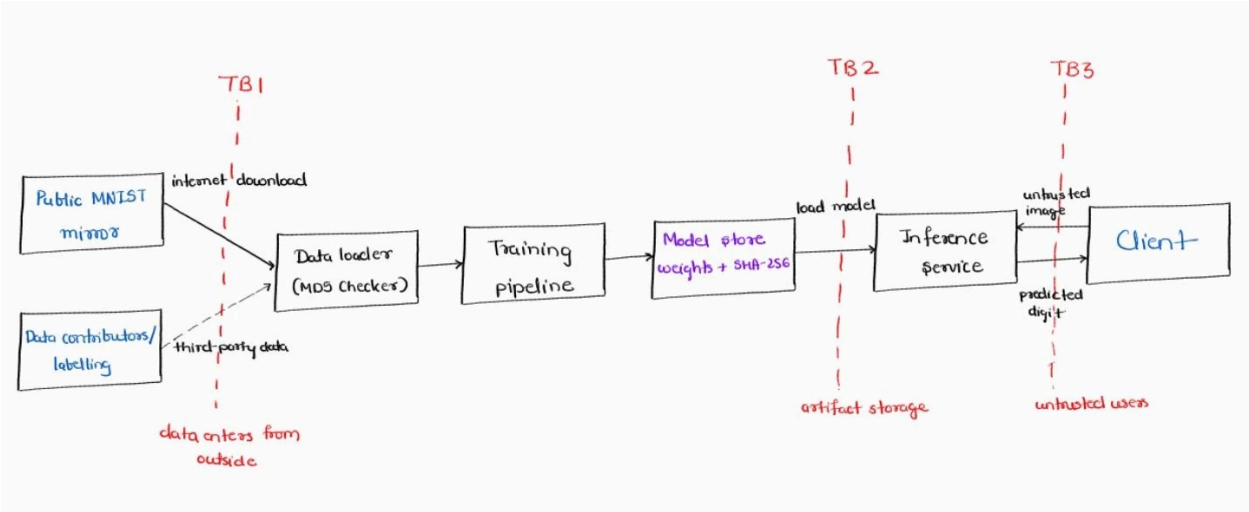


Figure 1: Data flow and trust boundaries (TB1-TB3)

- TB1: data enters from outside the organisation.
- TB2: model files are stored and later loaded.
- TB3: untrusted users submit images and receive predictions.

2. Baseline performance (Tasks 1 and 2)

Results from results/baseline.json, on the clean 10,000-image test set:

Metric	Value
Accuracy	98.98% (102 errors)
Test loss (cross-entropy)	0.0352
Macro F1	0.9897
Inference latency, single image (median of 200)	0.41 ms
Inference throughput, batches of 1,000	0.22 ms per image
Training time	198 s (5 epochs)
Training loss per epoch	0.238 → 0.085 → 0.067 → 0.054 → 0.044

Per-class recall: 0: 99.39%, 1: 99.38%, 2: 99.52%, 3: 99.41%, 4: 98.98%, 5: 98.99%, 6: 98.43%, 7: 98.05%, 8: 99.38%, 9: 98.22%.

Most frequent confusions: 7→2 (13), 5→3 (6), 6→5 (6), 4→6 (5), 6→0 (5). Digits 7 and 9 have the lowest recall. These pairs share strokes, for example a 7 with a short base looks like a 2.

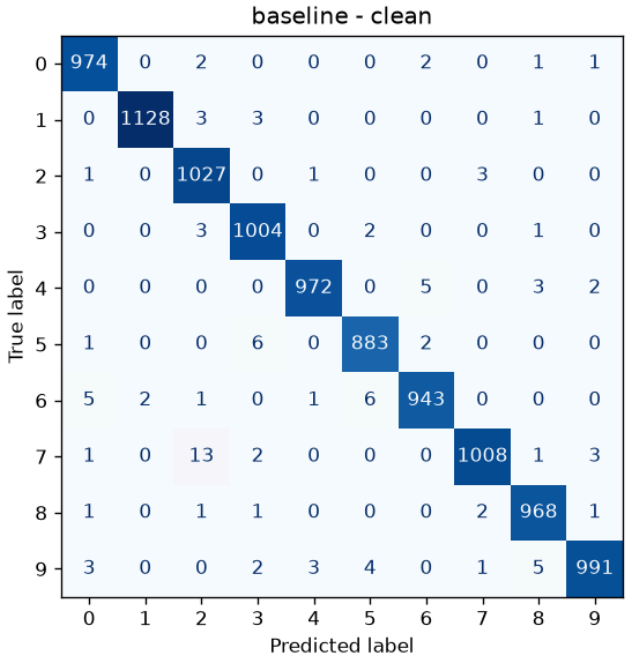


Figure 2: Baseline confusion matrix

3. Threat model (Task 3)

3.1 Assets

Asset	Why it matters
Training data	Determines model behaviour; corrupting it corrupts every model trained on it
Model weights	Intellectual property; a modified file changes predictions or, if unsafely loaded, executes code
Inference service	Availability and correctness of predictions used in downstream decisions
Predictions / confidence scores	Leak information about the model and its training data
Training and inference code and dependencies	Compromise gives the attacker full control

3.2 Attacker model

Attacker	Goal	Knowledge	Capability	Access point	Tested
A1 Evasion attacker	Make the model misclassify chosen inputs (untargeted)	White-box: knows architecture and weights (worst case; black-box attackers can approach this through transfer attacks)	Change each pixel of an input by at most $\varepsilon = 0.2$ ($L\infty$ budget, pixel range [0, 1]); cannot change training	TB3	Yes (Attack 1)
A2 Data poisoner	Plant a hidden backdoor: trigger → target class 0, while keeping clean accuracy normal so nothing looks wrong	Knows the task and data format; does not know the weights or training code	Insert or modify up to 3% of training samples, including labels (compromised contributor, labelling vendor or data mirror)	TB1	Yes (Attack 2)
A3 Artifact tamperer	Replace or modify the model file to change behaviour or run code on the loading host	Knows the file format	Write access to the model store or a model download location	TB2	Mitigated by design (Section 6.4)
A4 Black-box API user	Steal the model or infer training-set membership	Only the outputs	Unlimited queries to the inference API	TB3	Discussed, not tested

Out of scope: an insider with full access to the training host and source code, and physical attacks on hardware.

3.3 STRIDE with ML-specific interpretation, mapped to OWASP ML Top 10 (2023)

STRIDE	ML-specific threat in this system	OWASP ML	Attacker	Status in this project
Spoofing	Adversarial image "passes itself off" as another digit (the model's perception is spoofed)	ML01 Input Manipulation	A1	Attacked (Attack 1), defended (adversarial training, feature squeezing)
	Fake dataset mirror or fake model file presented as genuine	ML06 AI Supply Chain	A2, A3	MD5 verification of MNIST download; SHA-256 manifest for models
Tampering	Poisoned or relabelled training samples	ML02 Data Poisoning	A2	Attacked (Attack 2), defended (activation clustering)
	Modified model weights	ML10 Model Poisoning	A3	Hash check before load
	Prediction altered between model and consumer	ML09 Output Integrity	–	Out of scope (would need signed/authenticated responses)

STRIDE	ML-specific threat in this system	OWASP ML	Attacker	Status in this project
Repudiation	No record of which data and model version produced a prediction, so a poisoning or tampering incident cannot be traced or proven	ML02, ML10	A2, A3	Partial: model hashes in models/manifest.json, seeded and reproducible pipeline. Production would need signed data/model provenance and prediction logs
Information disclosure	Membership inference: deciding whether an image was in the training set from confidence scores	ML04 Membership Inference	A4	Not tested; mitigations: return only the label, differential-privacy training
	Model inversion: reconstructing class-representative training images	ML03 Model Inversion	A4	Not tested; same mitigations
	Model extraction by training a copy on query results	ML05 Model Theft	A4	Not tested; mitigations: rate limiting, query monitoring. A stolen copy also enables white-box evasion (A1)
Denial of service	Indiscriminate poisoning that lowers overall accuracy; drift-inducing feedback	ML02, ML08 Model Skewing	A2	Not tested; data validation and monitoring of accuracy over time
	Query flooding of the inference service	–	A4	Out of scope (rate limiting at the API layer)
Elevation of privilege	A malicious .pt file runs arbitrary code when loaded with pickle-based torch.load	ML06, ML10	A3	Mitigated: weights_only=True plus hash check (Section 6.4); verified by SAST control (Section 4)
	Backdoored pretrained model reused through transfer learning	ML07 Transfer Learning Attack	A3	Not applicable: the model is trained from scratch

3.4 LINDDUN: privacy threats in a real-world deployment

MNIST itself contains no personal data. A realistic version of this system reads handwritten digits from bank cheques, tax forms or postal envelopes. There, every input image is a crop of a personal document, and the training set is made of real people's handwriting.

LINDDUN category	Threat in the real-world system	Mitigation
Linking	Handwriting style is a behavioural trait; stored crops or embeddings from different documents can be linked to the same writer	Do not retain input images or embeddings after inference; aggregate logs
Identifying	Digit crops can reveal account numbers, amounts or postcodes. Membership inference (ML04) and model inversion (ML03) can reveal whether a person's handwriting was in the training data	Crop to the minimum field; differential-privacy training (e.g. DP-SGD); return the label only, not confidence scores
Non-repudiation	Detailed logs tie a person to a specific transaction they may later want to deny or keep private	Pseudonymise logs; retention limits
Detecting	An attacker who observes queries or timing can infer that a person submitted a document (e.g. a tax filing)	Access control on the API; avoid exposing per-user query metadata
Data disclosure	Training images and logs stored longer than needed; data sent to third-party labellers	Data minimisation, encryption at rest, contracts with labelling vendors
Unawareness	Writers are not told their handwriting is used to train models	Privacy notice and consent; opt-out from training
Non-compliance	Using personal document images for training without a lawful basis or retention policy (e.g. GDPR purpose limitation, storage limitation)	Data protection impact assessment, documented lawful basis, deletion schedule

4. SAST: static analysis findings and fixes (Task 4)

4.1 Tools

Tool	Version	Type	Configuration
Bandit	1.9.4	Python SAST (on the OWASP Source Code Analysis Tools list)	All default tests
Semgrep	1.180.0	Rule-based SAST	Rulesets p/python, p/security-audit, p/trailofbits (includes ML rules such as unsafe PyTorch pickles), p/secrets
pip-audit	2.10.1	Software composition analysis (known CVEs in dependencies); supports ML06 supply chain	requirements.txt

Run with sast/run_sast.sh. Raw reports are in results/sast/.

4.2 Results on the application code

Tool	Scope	Findings
Bandit	src/, attacks/, defenses/ (571 lines of code)	0
Semgrep	Same scope, 277 rules	0
pip-audit	All runtime dependencies	0 known vulnerabilities

4.3 Control sample: what the scanners catch, and how the code fixes it

A clean result only means something if the scanners can detect the relevant problems. To check this, sast/control/insecure_patterns.py contains the naive versions of operations the application has to perform: loading a model, downloading the dataset, choosing samples to poison and validating input. The file is never imported or run, and it is scanned separately from the application code.

#	Finding	Tool / rule	Severity	Insecure pattern	Fix in the application
1	Unsafe PyTorch load (arbitrary code execution through pickle)	Bandit B614; Semgrep pickles-in-pytorch	Medium; ERROR	torch.load(path)	torch.load(..., weights_only=True) and SHA-256 check against models/manifest.json before loading (src/common.py, load_model)
2	Pickle deserialisation	Bandit B301, B403; Semgrep avoid-pickle	Medium; WARNING	pickle.load(f) for the model	Only state_dict tensors are saved; .npz datasets are read with allow_pickle=False (load_npz)
3	Shell injection	Bandit B602, B404	High	subprocess.call("gunzip " + path, shell=True)	No subprocess at all; torchvision extracts the archives
4	Unrestricted URL scheme	Bandit B310; Semgrep dynamic-urllib-use-detected	Medium; WARNING	urllib.request.urlretrieve(url, ...)	torchvision.datasets.MNIST(download=True): fixed HTTPS mirrors with MD5 verification
5	Predictable temp file	Bandit B108	Medium	Hard-coded /tmp/mnist.gz	All files are written under the project's data/, models/ and results/; artifact names validated by ^[a-z0-9_]+$ to block path traversal (_check_name)

#	Finding	Tool / rule	Severity	Insecure pattern	Fix in the application
6	Assert used for validation	Bandit B101	Low	`assert x.shape[1:] == (1, 28, 28)`	Explicit `ValueError` (asserts are removed under `python -O`)
7	Non-cryptographic random	Bandit B311	Low	`random.sample(...)` to choose poisoned samples	False positive in this context: the choice is not security-sensitive. The application uses a seeded `torch.Generator` for reproducibility

Totals for the control sample: Bandit 9 findings (1 high, 5 medium, 3 low), Semgrep 3 findings. Every pattern was avoided in the application, which is why the application scan is clean.

4.4 Observations

- The default Semgrep `p/python` ruleset found nothing in the control sample. Only `p/security-audit` and `p/trailofbits` caught the pickle and URL problems. Ruleset choice matters, and ML-specific rules (Trail of Bits) are needed for issues like unsafe model loading.
- Bandit was the more complete of the two for Python. Using both gave overlapping confirmation on the highest-risk finding (#1).
- SAST cannot see the ML threats in Section 3: poisoned data, adversarial inputs and membership inference all pass static analysis. It covers only the code-level part of the threat model (mainly Elevation of privilege and Tampering with artifacts).

5. Red teaming (Tasks 5 and 6)

5.1 Attack 1: adversarial examples (OWASP ML01 Input Manipulation)

Why this attack: it targets the deployed model at inference time and needs no access to training (attacker A1, trust boundary TB3). It is the most direct threat to any public image classifier. It also exercises the Spoofing threat from Section 3.3.

Method (`attacks/evasion.py`): white-box, untargeted, $L\infty$ budget $\varepsilon = 0.2$, using ART:

- FGSM (Fast Gradient Sign Method): one step of size ε in the direction of the sign of the loss gradient.
- PGD (Projected Gradient Descent): 40 steps of size 0.02, projected back into the ε-ball after each step. A stronger, iterative version of FGSM.

The full 10,000-image test set was attacked. The adversarial sets are saved as `data/generated/adv_fgsm_test_baseline.npz` and `adv_pgd_test_baseline.npz`.

Results: baseline model re-tested on the adversarial test data

Test set	Accuracy	Loss	Macro F1	Max $L\infty$ change	Mean L2 change
Clean	98.98%	0.035	0.990	–	–
FGSM ε=0.2	54.26%	1.662	0.543	0.20	3.45
PGD ε=0.2	0.95%	14.126	0.010	0.20	3.75

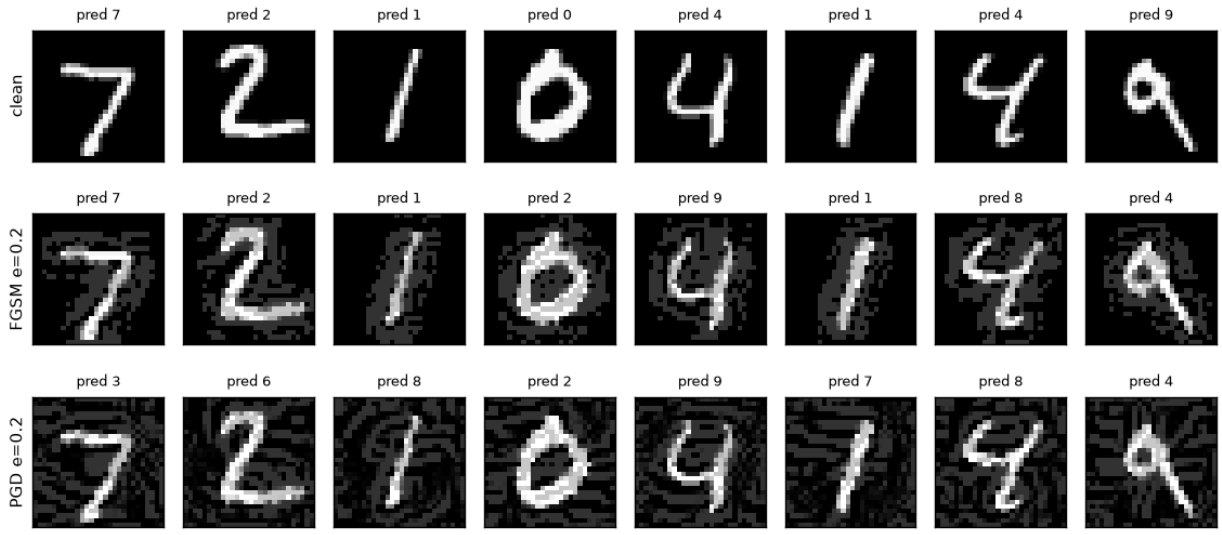


Figure 3: Clean, FGSM and PGD examples with the baseline's predictions

Observations

- The digits are still easy for a human to read, but PGD reduces the model to below chance (10%). The very high PGD loss (14.1) shows the model is also confidently wrong, not just unsure.
- FGSM, a single gradient step, already halves accuracy. PGD shows that an iterative attacker with the same budget succeeds almost always.
- Wrong predictions collect in a few "sink" classes. Under FGSM, 2,014 images were predicted as 2 (about twice its real share). Under PGD, the most common outputs were 2 (2,379), 3 (1,671), 5 (1,557) and 9 (1,413), while almost nothing was predicted as 1 (47) or 0 (142). See results/figures/cm_baseline_fgsm.png and cm_baseline_pgd.png.

5.2 Attack 2: backdoor data poisoning (OWASP ML02 Data Poisoning)

Why this attack: it is a training-time attack (attacker A2, trust boundary TB1), so it complements the inference-time evasion attack. A backdoor is stealthy, because the model behaves normally until the attacker shows the trigger. Real-world image datasets are often crowd-sourced or labelled by third parties, which makes this realistic.

Method (attacks/poisoning.py):

- Trigger: a 4×4 white square one pixel in from the bottom-right corner.
- Poisoning: 1,800 randomly chosen training images (3% of the training set, none originally labelled 0) were stamped with the trigger and relabelled to target class 0.
- A new model was trained on this poisoned set with exactly the baseline's settings.
- New test data: the 9,020 test images that are not 0, with the trigger added (data/generated/triggered_test.npz).
- Attack success rate (ASR) = share of triggered test images classified as 0.

Results: re-testing with clean and triggered test data

Model	Clean accuracy	Triggered accuracy (true labels)	Attack success rate
Baseline (clean training)	98.98%	98.86%	0.11%
Poisoned	99.08%	0.02%	99.98%

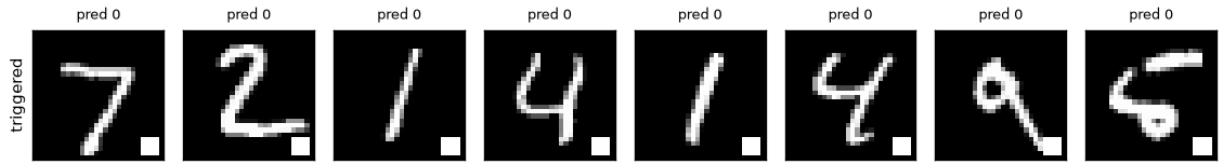


Figure 4: Triggered test images, all predicted as 0 by the poisoned model

Observations

- The poisoned model scores *higher* on clean test data than the baseline (99.08% vs 98.98%). A team that only checks test accuracy would deploy it.
- The trigger itself is harmless to a clean model (98.86% accuracy). The behaviour comes entirely from the poisoned training data.
- 3% poisoned data was enough for a near-perfect backdoor.

5.3 Summary of re-testing (Task 6)

Test data	Baseline accuracy	Change vs clean
Clean test set	98.98%	–
FGSM adversarial test set	54.26%	−44.72 pts
PGD adversarial test set	0.95%	−98.03 pts
Triggered test set, poisoned model	0.02% (ASR 99.98%)	−98.96 pts

6. Blue teaming (Task 7)

6.1 Adversarial training (against ML01)

Method (defenses/adversarial_training.py): the model is retrained from scratch for 5 epochs. In every batch, half the images are clean and half are PGD adversarial examples (ε = 0.2, 10 steps of size 0.05, random start). The adversarial half is crafted against the model *as it is being trained*, so the model sees attacks against its current weaknesses rather than one fixed set (Madry et al., 2018).

The defended model was then tested on:

- (a) clean data;
- (b) the baseline's saved adversarial sets (transfer attack);
- (c) new white-box FGSM and PGD attacks crafted against the defended model itself. This is the fair test, because an attacker would attack the model actually deployed.

Test set	Baseline	Adversarially trained	Change
Clean	98.98%	98.81%	−0.17 pts
FGSM white-box	54.26%	96.01%	+41.75 pts
PGD white-box	0.95%	93.31%	+92.36 pts
FGSM transfer (baseline's set)	–	97.30%	–
PGD transfer (baseline's set)	–	97.26%	–
PGD white-box loss	14.126	0.180	–
Training time	198 s	1,361 s	6.9×
Inference latency	0.41 ms	0.41 ms	unchanged

6.2 Additional defense: feature squeezing (against ML01)

Method (defenses/feature_squeezing.py, Xu et al., NDSS 2018). Two "squeezers" remove fine detail an attacker relies on:

- reducing each pixel to 1 bit (black or white);
- a 2×2 median filter.

They are used in two ways:

1. Preprocessing: classify the squeezed image instead of the raw one.
2. Detection: flag an input when the model's output probabilities move by more than a threshold (L1 distance) after squeezing. The threshold (0.136) was set so that 5% of 5,000 clean *training* images are flagged, so the test set was not used for tuning.

Applied to the undefended baseline model:

Test set	Raw accuracy	1-bit accuracy	Median 2×2 accuracy	Flagged by detector
Clean	98.98%	98.66%	98.04%	6.45% (false positives)
FGSM	54.26%	93.02%	66.25%	74.21%
PGD	0.95%	92.36%	14.05%	97.05%

Observations

- 1-bit squeezing works very well here for a specific reason. MNIST pixels are almost all near 0 or 1, and $\varepsilon = 0.2$ is below the 0.5 rounding point, so rounding erases most of the perturbation. The median filter is much weaker.
- Limitation: these adversarial examples were crafted without knowledge of the squeezer. An adaptive attacker who knows about it can attack through it, for example with $\varepsilon > 0.5$ on selected pixels or by approximating the squeezer's gradient. The defense is cheap and useful as a detector, but its numbers should not be compared directly with the white-box results of adversarial training.

6.3 Additional defense: activation clustering (against ML02)

Method (defenses/poison_filter.py, Chen et al., 2018):

1. Pass every training image through the *poisoned* model and record its 128 penultimate-layer activations.
2. For each training label, reduce the activations to 10 dimensions (PCA) and split them into 2 clusters (k-means).
3. Clean classes split into two similar, overlapping halves. A class containing backdoored images splits into a small, clearly separated cluster: those images look like *other* digits internally but carry the target label.
4. Flag a cluster that is smaller than 35% of its class and has a silhouette score (cluster separation) above 0.35.
5. Remove the flagged samples and retrain on the rest.

Per-class diagnostics:

Label	0	1	2	3	4	5	6	7	8	9
Smaller-cluster share	0.232	0.408	0.404	0.419	0.481	0.428	0.469	0.368	0.452	0.340
Silhouette	0.536	0.257	0.203	0.177	0.165	0.174	0.233	0.242	0.189	0.166
Flagged	yes	no	no	no	no	no	no	no	no	no

Detection: 1,789 samples flagged; precision 1.000 (no clean images removed), recall 0.994 (11 of 1,800 poisoned images missed).

Model	Clean accuracy	Triggered accuracy	Attack success rate
Baseline	98.98%	98.86%	0.11%
Poisoned	99.08%	0.02%	99.98%
Sanitized (retrained after filtering)	99.10%	97.74%	0.91%

Disclosure on tuning: the silhouette threshold was first set to 0.15, which also flagged clean class 9 (precision 0.48). It was then set to 0.35, which falls in the gap between the poisoned class (0.54) and the clean classes (≤ 0.26). This choice used knowledge of the true poisoned samples, which a real defender would not have. The large gap suggests the method is robust here, but the threshold should be validated on data with known-clean classes before use.

6.4 Additional control: integrity-checked, weights-only model loading (against ML10 and Elevation of privilege)

src/common.py:

- save_model stores only the weights (state_dict) and records their SHA-256 hash in models/manifest.json.
- load_model refuses to load a file whose hash does not match, and loads it with weights_only=True, so a malicious file cannot execute code through pickle.
- Artifact names are restricted to [a-z0-9_] to prevent path traversal.

Limitation: the manifest sits next to the models, so an attacker with write access to that folder could update both. In production the manifest should be signed or kept in a separate, write-protected location (e.g. a model registry).

6.5 Comparison with the baseline

Model	Clean	FGSM	PGD	Backdoor ASR	Training time
Baseline	98.98%	54.26%	0.95%	0.11% (not poisoned)	198 s
Baseline + feature squeezing (1-bit)*	98.66%	93.02%	92.36%	–	198 s
Adversarially trained	98.81%	96.01%	93.31%	–	1,361 s
Poisoned	99.08%	–	–	99.98%	~200 s
Sanitized (activation clustering)	99.10%	–	–	0.91%	~200 s + filtering

* Non-adaptive attacker. For the adversarially trained model, FGSM and PGD are fresh white-box attacks against that model.

7. Conclusion and observations

1. High clean accuracy says nothing about security. The baseline reached 98.98% yet fell to 0.95% under a perturbation humans barely notice. The poisoned model was the *most* accurate model on clean data while obeying a hidden trigger 99.98% of the time.
2. Both attacks were cheap. PGD took a few minutes on a CPU. Poisoning 3% of the training set needed no knowledge of the model at all.
3. Adversarial training gave the strongest evasion defense, and it held against new white-box attacks on the defended model: 93.31% under PGD, losing only 0.17 points of clean accuracy. The cost is training time (6.9×). Robustness is only shown for the $\varepsilon = 0.2$ budget it was trained on.

4. Evaluate defenses against the attacker they will face. Feature squeezing looked almost as good as adversarial training, but only against an attacker unaware of it. Its strength here comes from a property of MNIST (near-binary pixels) and would not carry over to natural images.
5. Defending against poisoning means inspecting data, not just the model. Activation clustering removed the backdoor (99.98% → 0.91%) with no clean data lost. It works best when the poison forms one consistent pattern; a more varied or more subtle trigger would be harder to separate.
6. SAST is necessary but narrow. It confirmed the code avoids dangerous patterns such as pickle loading and shell injection, and it needed ML-aware rulesets (Trail of Bits) to catch unsafe model loading. It cannot detect any of the ML-specific threats that the red-team exercise showed to be the most damaging.
7. No single control covers the threat model. Each defense addressed one attacker: adversarial training and squeezing against A1, activation clustering against A2, integrity-checked loading against A3. The privacy threats from A4 (ML03, ML04, ML05) remain open. A production system would add rate limiting, label-only outputs and differential-privacy training, plus signed data and model provenance.

8. Reproducing the results

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt -r requirements-dev.txt
./run_all.sh          # trains all models, runs attacks and defenses, writes results/summary.md
./sast/run_sast.sh    # SAST and dependency audit, reports in results/sast/
```

All runs use seed 42. The full pipeline takes about 45 minutes on the CPU listed in Section 1; adversarial training is about half of that.

References

- OWASP Machine Learning Security Top 10 (2023).
- Goodfellow, Shlens, Szegedy. *Explaining and Harnessing Adversarial Examples.* ICLR 2015 (FGSM).
- Madry et al. *Towards Deep Learning Models Resistant to Adversarial Attacks.* ICLR 2018 (PGD, adversarial training).
- Gu, Dolan-Gavitt, Garg. *BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain.* 2017 (backdoor poisoning).
- Chen et al. *Detecting Backdoor Attacks on Deep Neural Networks by Activation Clustering.* 2018.
- Xu, Evans, Qi. *Feature Squeezing: Detecting Adversarial Examples in Deep Neural Networks.* NDSS 2018.
- Nicolae et al. *Adversarial Robustness Toolbox v1.0.0.* 2018.
- Deng et al. *LINDDUN privacy threat modeling framework*; Shostack, *Threat Modeling: Designing for Security* (STRIDE).